

ELEC0520

Python programming

Summer 2026

연산자와 조건문

https://colab.research.google.com/drive/1_nXEkkI0SMHWI1jxBUG6mCHUyzGRAkiO?usp=sharing

자료형과 연산자

수식

자료형

연산자

산술

할당

관계

논리

Goal

- 사용자로부터 하나의 정수를 입력 받아 0 초과 100 미만의 짝수 인지를 판단하는 프로그램

정수: 40

입력된 정수는 0에서 100 사이에 있는 짝수입니다: True

정수: -40

입력된 정수는 0에서 100 사이에 있는 짝수입니다: False

정수: 35

입력된 정수는 0에서 100 사이에 있는 짝수입니다: False

- 세 자리 정수를 입력 받아 역순으로 출력하는 프로그램

세자리 정수를 입력하세요: 591

195

표현식 **expression**

- 수식이라고도 함
- 피연산자와 연산자의 조합
- 피연산자에 따라 제공되는 연산자가 달라짐

표현식은 연산자와 피연산자의 조합



자료형 `data type`

- 숫자
 - 정수 `int`
 - 실수 `float`
 - 복소수 `complex`
- 부울 `bool`
- 문자열 `str`
- 리스트 `list`, 튜플 `tuple`, 딕셔너리 `dict`, 등...
- `type()` 함수를 통해 확인 가능

산술 연산자

연산자	의미	동작
+	덧셈	왼쪽 피연산자와 오른쪽 피연산자를 더한다.
-	뺄셈	왼쪽 피연산자에서 오른쪽 피연산자를 뺀다.
*	곱셈	왼쪽 피연산자와 오른쪽 피연산자를 곱한다.
/		왼쪽 피연산자를 오른쪽 피연산자로 나눈다. 파이썬의 나눗셈은 기본적으로 실수값을 반환한다.
//		/ 와 달리 나눗셈의 결과를 소수점 이하를 버리고 정수 부분만을 얻고자 할 경우에 사용한다.
%		모듈로 연산자라고 읽으며 비율을 의미하는 퍼센트와는 상관이 없다. 나눗셈의 나머지를 구한다.
**		왼쪽 피연산자를 오른쪽 피연산자로 거듭제곱한다.

```
>>> 14 / 5
```

```
2.8
```

```
>>> 14 // 5
```

```
2
```

```
>>> 14 % 5
```

```
4
```

```
>>> 14.2 // 5.3      # 14.2를 5.3으로 나눈 몫
```

```
2.0
```

```
>>> 14.2 % 5.3      # 14.2을 5.3으로 나눈 나머지
```

```
3.5999999999999996
```

```
>>> 14.2 - (5.3 * 2.0)  # 14.2 // 5.3 으로 구한 몫이 2.0이므로 나머지는 이런 의미
```

```
3.5999999999999996
```

- 5를 2로 나눈 나머지
- 제곱근 2, 제곱근 3

대화창 실습 : 부동소수점 수의 수치오류

```
>>> print(0.1 + 0.1 == 0.2)
```

```
True
```

```
>>> print(0.1 + 0.1 + 0.1 == 0.3)
```

```
False
```

```
>>> 0.1 + 0.1 + 0.1
```

```
0.30000000000000004
```

예상 출력은 0.3

예상 출력은 0.3이지만 실제로는 아래와 같음

```
>>> 0.1 + 0.1 + 0.1 + 0.1
```

```
0.4
```

```
>>> 0.1 + 0.1 + 0.1 + 0.1 + 0.1
```

```
0.5
```

```
>>> 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 # 예상 출력은 0.8
```

```
0.7999999999999999
```

```
>>> 0.1 * 8
```

```
0.8
```

```
>>> 0.1 * 19.0
```

```
1.9000000000000001
```

여기서도 마찬가지로 현상
컴퓨터의 실수 값은 저장될 때 미세한 수치 오류를
포함하고 있음.

실수 표현의 오류

복소수complex

```
>>> c1 = 2 + 3i
```

```
c1 = 2 + 3i
```

```
^
```

SyntaxError: invalid syntax

```
>>> c1 = 2 + 3j
```

```
>>> c1.real      # c1의 실수부 출력
```

```
2.0
```

```
>>> c1.imag      # c1의 허수부 출력
```

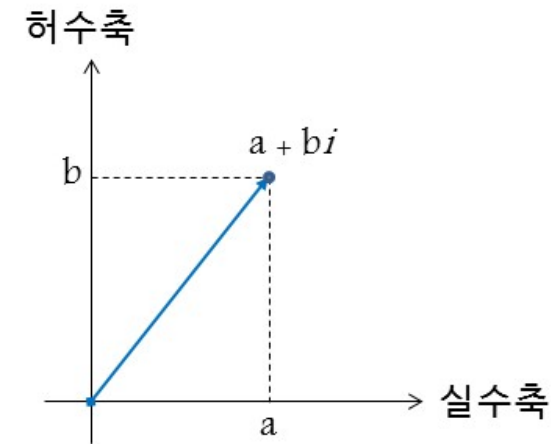
```
3.0
```

```
>>> c2
```

```
(2-3j)
```

```
>>> abs(c1)
```

```
3.605551275463989
```



할당 연산자 assign operator

- 우변의 값을 좌변의 변수에 대입
- 다중 할당
- **동시 할당**
- 복합대입연산자

두 변수의 값 교환(**swap**)

`a, b = b, a`

대화창 실습 : 다중 할당과 동시 할당

```
>>> num1 = num2 = num3 = 200    # 다중 할당문
```

```
>>> print(num1, num2, num3)
```

```
200 200 200
```

```
>>> num4, num5 = 300, 400      # 동시 할당문
```

```
>>> print(num4, num5)
```

```
300 400
```

대화창 실습 : 할당 연산 실습

```
>>> result1 = 10 * 20
```

```
>>> result1
```

```
200
```

```
>>> result2 = (2 * 3) - (4 ** 2) / 2
```

```
>>> result2
```

```
-2.0
```

```
>>> 300 = 300 # 등호는 두 값이 같다는 의미가 아님
```

```
...
```

```
SyntaxError: can't assign to literal
```

```
>>> str = 'world'
```

```
>>> 'hello' = str # 리터럴에는 변수를 할당할 수 없다
```

```
...
```

```
SyntaxError: can't assign to literal
```

```
>>> num = 200
```

```
>>> num = num + 100 # 200 + 100 연산을 수행하여 그 결과를 num에 할당
```

```
>>> num
```

```
300
```

연산자

+=

--

*=

/=

//=

%=

**=

관계 연산자

비교연산자	설명	a = 100, b = 200일 때
==	두 피연산자의 값이 같으면 True를 반환한다.	a == b는 False
!=	두 피연산자의 값이 다르면 True를 반환한다.	a != b는 True
>	왼쪽 피연산자가 오른쪽 피연산자보다 클 때 True를 반환한다.	a > b는 False
<	왼쪽 피연산자가 오른쪽 피연산자보다 작을 때 True를 반환한다.	a < b는 True
>=	왼쪽 피연산자가 오른쪽 피연산자보다 크거나 같을 때 True를 반환한다.	a >= b는 False
<=	왼쪽 피연산자가 오른쪽 피연산자보다 작거나 같을 때 True를 반환한다.	a <= b는 True

대화창 실습 : 비교 연산 (비교의 결과가 True, False로 출력된다)

```
>>> a, b = 100, 200
```

```
>>> a == b
```

```
False
```

```
>>> a != b
```

```
True
```

```
>>> a > b
```

```
False
```

```
>>> a < b
```

```
True
```

```
>>> a >= b
```

```
False
```

- 주의할 점은 !=에서 !와 =사이에 공백을 넣으면 안되며, >= 연산에서도 >와 = 사이에 공백을 넣으면 안된다.
- =>와 같이 등호와 > 연산자의 순서가 바뀌어도 에러가 뜬다.

부울bool

```
>>> bool(-1)
True
>>> bool(0)
False
>>> bool(None)
False
>>> bool("")
False
>>> bool('hello')
True
>>> bool([])
False
>>> bool([10, 20])
True
```

- True(참)/False(거짓) 값을 가짐
 - 0, 빈 값 -> False로 간주
 - non-zero, 값 있는 자료 -> True로 간주
- 파이썬에서 **그 자체로 참인 값**
 - True, 1, 1.0
- 파이썬에서 **그 자체로 거짓인 값**
 - False, 0, 0.0

논리 연산자logical operator

- and, or, not
- 결과는 bool 타입: True/False

대화창 실습 : 논리 연산 실습

```
>>> x = True
```

```
>>> y = False
```

```
>>> x and y
```

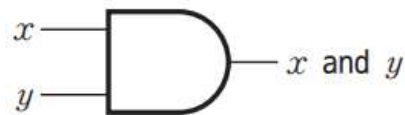
```
False
```

```
>>> x or y
```

```
True
```

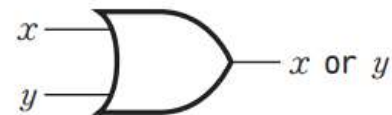
```
>>> not x
```

```
False
```



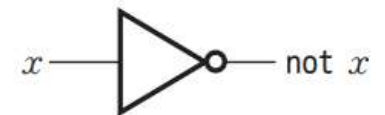
x and y

x	y	x and y
False	False	False
False	True	False
True	False	False
True	True	True



x or y

x	y	x or y
False	False	False
False	True	True
True	False	True
True	True	True



not x

x	not x
False	True
True	False

문자열str

- 작은 따옴표('), 큰 따옴표(")를 통해 문자열을 묶음
 - 따옴표 3개 사용하면 여러 줄도 가능
- 문자열 구성하기
 - "친구가 "해님이 좋아!"라고 말했다"라는 문자열은?
 - 문자열 내부에 따옴표가 포함된 경우
 - 제어escape문자
 - 개행
 - 탭
- +, * 연산자 사용 가능

in 연산자

- 특정한 원소가 문자열, 리스트, 튜플과 같은 컬렉션 내부에 포함되어 있는가를 검사하는 용도로 사용됨
- 참(True) 혹은 거짓(False)를 반환
- 원소 in 컬렉션
- not in: 포함되어 있지 않은지 확인

```
>>> lst = [1,2,3,]  
>>> 3 in lst  
True  
>>> 3 not in lst  
False  
>>> st = 'python'  
>>> 'p' in st  
True  
>>> 'Y' not in st  
True
```

print() 함수의 디폴트 인자

- print() 함수의 매개변수

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

- sep: 인자들 간의 출력 제어
- end: 마지막 인자 출력 제어

```
>>> print('hello', 'python')
hello python
>>> print('hello', 'python', sep = ', ')
hello, python
>>> print('hello', 'python', end = '.')
hello python.>>> print('hello', 'python', sep = ', ', end = '.')
hello, python.>>>
```

조건문

if

if-else

if-elif-else

조건문 활용

- 놀이공원 입장 프로그램

- 나이를 입력 받아 20세 미만이면 '청소년 할인' 추가 출력

나이: 21
입장하세요.

나이: 15
청소년 할인되었습니다.
입장하세요.

- 1000걸음 이상이면 '목표 달성'을 출력하는 프로그램

걸음 수: 51

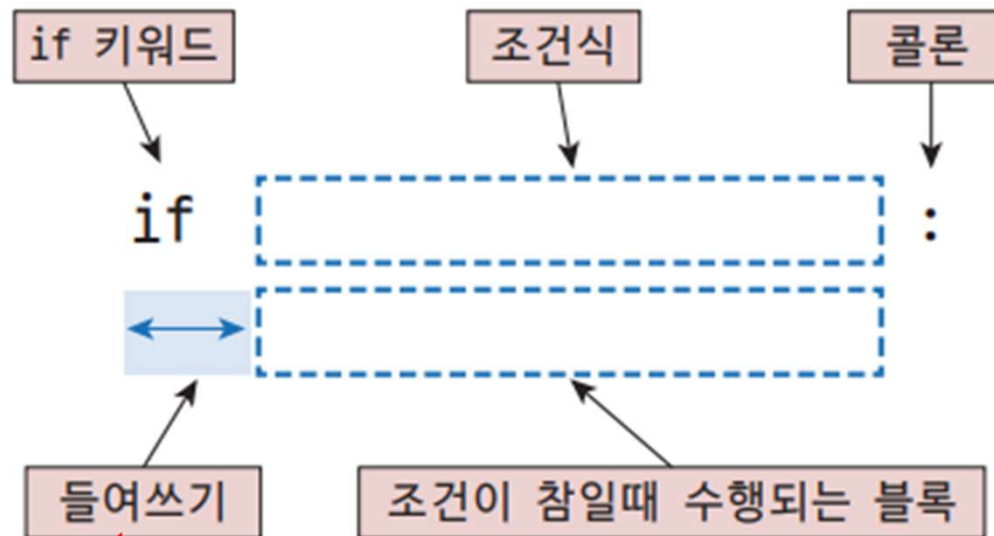
걸음 수: 1000
목표 달성

- 시간을 입력 받아 12시 전이라면 '오전', 12시 이후라면 '오후'를 출력하는 프로그램

시간: 10
오전입니다.

시간: 23
오후입니다.

조건문



파이썬은 들여쓰기가 아주 중요하다

조건문 활용 2

- 속도를 입력 받아 100km/h 이상이면 '고속', 60km/h 이상 100km/h 미만이면 '중속', 60km/h 미만이면 '저속'을 출력하는 프로그램
- 점수를 입력 받아 점수에 따라 아래와 같이 등급을 출력하는 프로그램

점수	등급
100점 ~ 90점 이상	A
90점 미만 ~ 80점 이상	B
80점 미만 ~ 70점 이상	C
70점 미만 ~ 60점 이상	D
60점 미만	F

입력한 달에 존재하는 날의 수 출력 코드

- 31일
 - 1, 3, 5, 7, 8, 10, 12월
- 30일
 - 4, 6, 9, 11월
- 2월은??
 - 윤년 여부에 따라 다름
 - 윤년 : 29일
 - 년도가 4의 배수 but 100의 배수는 아님
 - 또는 년도가 400의 배수
 - 평년 : 28일

년도: 2022

달: 3

2022년 3월은 31까지 존재합니다.

년도: 2022

달: 2

2022년 2월은 28까지 존재합니다.

년도: 2022

달: -5

[입력오류] 1~12월만 존재합니다.

문자 분류하기

- 사용자로부터 문자 하나를 입력 받아, 숫자, 알파벳 모음, 자음, 기타문자로 분류하는 프로그램

문자 하나를 입력해주세요: A
A는 알파벳 모음입니다.

문자 하나를 입력해주세요: b
b는 알파벳 자음입니다.

문자 하나를 입력해주세요: 9
9는 숫자입니다.

문자 하나를 입력해주세요: +
+는 기타문자입니다.

문자열 메소드

- dir(str)

```
>>> dir(str)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```